Fady Youssef
11/2/2025
CPSC 321, Fall 2025

HW6

---

1. Order(orderID, productID, customerID, customerName, ProductName, price)

orderID	productID	customerID	customerName	productName	price
1001	354	3245	Alvin Poole	Headphones	$39.99
1001	432	3245	Alvin Poole	Laptop	$899.99
1002	354	7585	Kyra Rush	Headphones	$39.99
1003	231	3866	Lea Fry	Mouse	$24.99

(a) **Functional Dependencies:**
orderID → customerID, customerName
- An order placed is for only 1 customer

productID → productName, price
- every specific product has a singular specific name and price

customerID → customerName

(b) For every time a customer places an order, the orderID, customerName, and customerID show up for each product order causing major redundancy. Furthermore, since we have customerID, customerName is redundant in this relation since we already know who they are through the customerID.

(c) **Update Anomaly:**
If a customer decides to change their name or a provider decides to change their price for a product for instance, we would need to find a way to update that field for every single row or else there will be inconsistencies.

Insertion Anomaly:
We cannot add new customers to a database until they place an order because even if we try, all other columns would be set to NULL along with violating the primary key constraint.

Deletion Anomaly:

If we decide to delete an order from the database then that order gets cancelled, we lose all of the information about that order like productName, price, and productID even though we would still want it listed.

2. **Provide Direct Proofs for the following three rules:**

(a) Union: if $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

$XX \rightarrow XY$

$X \rightarrow XY$

$XY \rightarrow YZ$

Therefore, by the transitive property: $X \rightarrow YZ$

(b) Decomposition: if $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

Given $X \rightarrow YZ$

We know $YZ \rightarrow Y$

And $YZ \rightarrow Z$ through the reflexive property

Therefore by the transitive property $X \rightarrow Y$ and $X \rightarrow Z$

(c) Pseudotransitivity: if $X \rightarrow Y$ and $YW \rightarrow Z$, then $XW \rightarrow Z$

Given $X \rightarrow Y$ and $YW \rightarrow Z$

$XW \rightarrow YW$ through augmentation

Then by the transitive property $XW \rightarrow Z$

3. **Find all of the candidate keys for each the following scenarios:**

(a) relation R(a,b,c,d) and FDs $\{b \rightarrow c, d \rightarrow a\}$

$bd \rightarrow b$

$bd \rightarrow c$ through reflexivity

$bd \rightarrow b$ (from $b \rightarrow c$) through augmentation

$bd \rightarrow a$ (from $d \rightarrow a$) through augmentation

therefore , $bd \rightarrow abcd$

Candidate key: $\{b,d\}$

b of course returns b along with c while d return d and a, without both b and d we can't get all the relations therefore bd as a candidate key is minimal.

(b) relation R(a,b,c,d,e) and FDs $\{a \rightarrow bc, cd \rightarrow e, b \rightarrow d, e \rightarrow a\}$

$a \rightarrow bc$

$a \rightarrow b$ and $a \rightarrow c$ through decomposition

$b \rightarrow d$

$a \rightarrow abcd$

$cd \rightarrow e$ so through transitivity $a \rightarrow e$

Therefore a → abcde

Candidate key: {a}, {e}, {c,d}

Since a returns the other relations including e and e returns a, a and e are the candidate keys because they return all other relations. Finally since c,d → a and we already know a → abcde then {c,d} → abcde

(c) relation R(a,b,c,d,e) and FDs {c → b, bd → e, a → d, e → a}

ac → bd

bd → e

Therefore through transitive property ac →e

Leaving ac →abcde

But since e → a

ec → abcde

Candidate Keys: {a,c}, {c,e}, {c,d}

And again, since cd → a, then cd → abcde making {c,d} another candidate key.

4. **state the candidate key(s) for the table and the highest normal form each table is:**

(a) movie_info′(director, studio, studio_loc)

Director → studio (one director per studio)

Studio → studio_loc (one location per studio)

Director → studio_loc (through transitive property)

Therefore director → director, studio, studio_loc

Candidate key: director

Normal Form: 2NF; because studio_loc doesn't directly depend on director rather it depends on studio therefore violating 3NF

(b) movie_info′(title, director, studio)|

Candidate key: {title,director}

Normal Form: 1NF; studio depends on the director instead of the entire candidate key which violates 2NF because the functional dependency doesn't depend on the entire key.

(c) movie_info′(award_type, award_year, studio_loc)

Candidate key: {award_type, award_year, studio_loc}

Normal Form: BCNF; no non-key attributes, no partial dependencies therefore this is in BCNF

5. relation R(a, b, c, d, e) with FDs {a → bc, cd → e, b → d, e → a}

(a) Show that the decomposition R1(a, b, c) and R2(a, d, e) is lossless.

a →abc
b →d
a → abcd
cd → e
e → a
a → abcde

This decomposition is lossless because attribute ‘a’ superkey present in both relations.

(b) Show that the decomposition is not dependency preserving.

For R_1 a → bc

But for R_2 we need c and d for e which c is not in R_2

Also, we need b for attribute d (b →d) which is not in R_2

Therefore the decomposition is not dependency preserving because b →d and cd →e can’t be verified in either relation alone requiring both relations joining.

6. Give a lossless and dependency preserving decomposition of the full (original) movie_info relation using the BCNF Algorithm. Draw the decomposition as a tree.

movie_info(award_type, award_year, title, director, studio, studio_loc)
Key: {award_type, award_year}
FDs: {award_type, award_year → title, director
studio → studio_loc
director → studio}

studios(studio, studio_loc)
Key: {studio}
FDs: {studio → studio_loc}

movie_info_1(award_type, award_year, title, director, studio)
Key: {award_type, award_year}
FDs: {award_type, award_year → title, director
director → studio}

directors(director, studio)
Key: {director}
FDs: {director → studio}

movie_info_2(award_type, award_year, title, director)
Key: {award_type, award_year}
FDs: {award_type, award_year → title, director}